

Prof. Stefan Göller
Anna Maiworm

Einführung in die Informatik

WiSe 2025/2026

Hausaufgabenblatt 06, Abgabe am 08.12.25 um 12:00 Uhr

Allgemeine Hinweise

- Wenn die Hausaufgabe exakte Benennungen von Dateien, Funktionen, Variablen, etc. vorgibt, dann müssen Sie sich an diese Vorgabe halten. Falls nicht wird Ihre Abgabe nicht oder nur mit Punktabzügen korrigiert.
- Gleiches gilt, wenn die Aufgabe konkrete Anweisungen für `input()` oder `print()` vorgibt.
- Ihr Programm darf keine zusätzlichen `print()` Befehle enthalten, welche in der Aufgabe nicht gefordert sind. Ist eine Ausgabe mit `print()` gefordert, so geben Sie, sofern nicht anders vorgegeben, nur das geforderte Ergebnis und **keinen** zusätzlichen Text aus.
- Benutzen Sie **keine Python-Imports**, außer dies ist in der Aufgabenstellung explizit erlaubt.
- Beachten Sie die Anweisungen, die ggf. zusätzlich in den jeweiligen Aufgaben gegeben sind.
- Geben Sie für jede Aufgabe eine separate Datei ab. Benennen Sie diese Datei als `bXXaY.py`, wobei XX die Blattnummer und Y die Aufgabennummer ist.
- Schreiben Sie die Namen aller Gruppenmitglieder als Kommentar in die erste Zeile jeder Datei.
- Programmcode muss möglichst einfach und gut lesbar sein, um die Korrektur zu erleichtern. Code, der nicht oder nur schwer lesbar ist, kann zu Punktabzug führen.

Hausaufgabe 1 (3 Punkte):

Schreiben Sie eine Funktion `gerade_summe`, welche als Eingabe einen Integer $n \geq 1$ erwartet und rekursiv die Summe der ersten n geraden natürlichen Zahlen berechnet und zurückgibt. Hierbei zählen wir die 2 als die erste gerade Zahl.

Beispiel: Die Eingabe 5 führt zur Rückgabe 30, denn $2 + 4 + 6 + 8 + 10 = 30$.

Hausaufgabe 2 (3 Punkte):

Schreiben Sie eine Funktion `zerfall`, welche drei Eingaben erwartet:

- eine positive Zahl n (Startwert)
- einen Integer-Wert p zwischen 0 und 100 (Prozentangabe)
- einen nicht-negativen Integer-Wert t (Anzahl der Tage)

Die Funktion soll mittels Rekursion das folgende berechnen und das Ergebnis immer als Float zurückgeben:

Angenommen wir haben zu Beginn n Einheiten einer Materie und jeden Tag zerfallen p Prozent davon. Wie viele Einheiten sind dann nach t Tagen noch vorhanden?

Beispiele:

- `zerfall(200, 50, 0)` liefert die Rückgabe 200.0
- `zerfall(200, 50, 1)` liefert die Rückgabe 100.0
- `zerfall(200, 50, 4)` liefert die Rückgabe 12.5
- `zerfall(180.9, 17, 2)` liefert die Rückgabe 124.62200999999999

Hinweis: Rundungsfehler, wie im letzten Beispiel angegeben, können bei Ihnen auftreten, müssen es aber nicht.

Hausaufgabe 3 (4 Punkte):

Schreiben Sie eine Funktion **zerlegung**, welche als Eingabe einen String **w** erwartet. Die Funktion soll mittels Rekursion eine Liste von Strings der Länge 2 berechnen, welche die Eingabe **w** zerlegt darstellt. Bleibt ein Buchstabe übrig, so enthält die Rückgabeliste als letzten Eintrag einen String der Länge 1.

Beispiele:

- Die Eingabe "abcdef" führt zur Rückgabe ["ab", "cd", "ef"].
- Die Eingabe "abc" führt zur Rückgabe ["ab", "c"].
- Die Eingabe "" führt zur Rückgabe [].

Hausaufgabe 4 (3 Punkte):

Schreiben Sie eine Funktion **enthaelt**, welche als Eingabe eine Liste und eine weitere Eingabe **element** erwartet. Die Funktion soll mittels Rekursion bestimmen, ob **element** als Eintrag in der Liste vorkommt. Falls ja soll **True** ausgegeben werden, sonst **False**.

Beispiele:

- Gibt man [2, "b", 9] und 2 ein, dann erhält man die Rückgabe **True**.
- Gibt man [2, [1], [2]] und [1] ein, dann erhält man die Rückgabe **True**.
- Gibt man [2, "ab", 9] und "a" ein, dann erhält man die Rückgabe **False**.
- Gibt man [1, [[1]], [1, [1]]] und [1] ein, dann erhält man die Rückgabe **False**.

Hausaufgabe 5 (4 Punkte):

Schreiben Sie eine Funktion `filter`, welche als Eingabe eine Liste und eine Bedingung `bedingung` erwartet, d.h. eine Funktion, welche wiederum irgendeine Eingabe erwartet und einen booleschen Wert zurückgibt.

Die Funktion `filter` soll mittels Rekursion die Liste berechnen, die man erhält, wenn man aus der Eingabeliste nur diejenigen Einträge `e` auswählt, für die `bedingung(e)` den Wert `True` zurückgibt.

Sie dürfen davon ausgehen, dass Ihre Funktion nur mit Eingaben aufgerufen wird, bei denen `bedingung` auf die Einträge der eingegebenen Liste anwendbar ist.

Beispiel: Enthält das Hauptprogramm neben Ihrer Funktion noch den folgenden Code ...

```
def test_bed (zahl):  
    return zahl < 10
```

... dann liefert `filter([19,-2,10,9],test_bed)` die Rückgabe `[-2,9]`.

Hausaufgabe 6 (3 Punkte):

Schreiben Sie eine Funktion `anfang_von`, welche zwei Tupel beliebiger Größen $m, n \geq 0$ erwartet und rekursiv berechnet, ob beide Tupel in den ersten $\min(m, n)$ Einträgen übereinstimmen. Falls ja soll `True` ausgegeben werden, sonst `False`.

Beispiele:

- Gibt man `(2, "b", 9, 100)` und `(2, "b", 9)` ein, dann erhält man die Rückgabe `True`.
- Gibt man `(2, "b", 9)` und `(2, "b", 9, 100)` ein, dann erhält man die Rückgabe `True`.
- Gibt man `(2, "b", 9)` und `(2, 9)` ein, dann erhält man die Rückgabe `False`.

Präsenzaufgaben

Präsenzaufgabe 1:

Schreiben Sie eine Funktion `listen_summe`, welche als Eingabe eine Liste von Integer-Werten erwartet. Die Funktion soll mittels Rekursion die Summe der Listeneinträge berechnen und diese zurückgeben.

Beispiele:

- Die Eingabe `[44,26,-15,60]` führt zur Rückgabe 115.
- Die Eingabe `[]` führt zur Rückgabe 0.

Präsenzaufgabe 2:

Schreiben Sie eine Funktion `verdoppeln`, welche als Eingabe einen String erwartet und mittels Rekursion den String berechnet der entsteht, wenn man jedes Zeichen im Eingabestring verdoppelt. Zurückgegeben werden soll der berechnete String.

Beispiel: Die Eingabe `"a24X"` führt zur Rückgabe `"aa2244XX"`.

Präsenzaufgabe 3:

Schreiben Sie eine Funktion `verknuepfen`, welche zwei Tupel `t1`, `t2` der selben Länge erwartet. Die Funktion soll die beiden Tupel mittels Rekursion zu einem Tupel verknüpfen, indem abwechselnd Zeichen aus `t1` und `t2` eingefügt werden. Das resultierende Tupel soll zurückgegeben werden.

Beispiel: Gibt man `(1,2,3)` und `("a","b","c")` ein, dann erhält man die Rückgabe `(1,"a",2,"b",3,"c")`.

Präsenzaufgabe 4:

Schreiben Sie eine Funktion `ist_anagramm`, welche als Eingabe zwei Strings `w`, `v` erwartet. Die Funktion soll mittels Rekursion berechnen, ob `w` ein Anagramm von `v` ist, d.h. ob `w` und `v` aus den gleichen Zeichen bestehen und jedes Zeichen gleich oft vorkommt. Falls ja soll `True` ausgegeben werden, sonst `False`.

Beispiele:

- Gibt man "ABLEGER" und "GELABER" ein, dann erhält man die Rückgabe `True`.
- Gibt man "Ableger" und "Gelaber" ein, dann erhält man die Rückgabe `False`.
- Gibt man "AFFE" und "AFEE" ein, dann erhält man die Rückgabe `False`.
- Gibt man "aaa" und "aa" ein, dann erhält man die Rückgabe `False`.

Hinweis: Ist `z` ein Zeichen (String der Länge 1) und `s` ein String, dann liefert `s.replace(z, "", 1)` einen neuen String, der aus `s` entsteht, indem das erste Vorkommen von `z` entfernt wird. Solange das Zeichen tatsächlich enthalten ist bewirkt diese Anweisung also das gleiche auf Strings, was `L.remove(z)` auf einer Liste `L` bewirkt.

Präsenzaufgabe 5:

Schreibe eine Funktion `teilmengen`, welche als Eingabe eine Liste erwartet, die eine Menge repräsentiert, d.h. kein Eintrag kommt doppelt vor. Die Funktion soll mittels Rekursion die Menge aller Teilmengen berechnen und diese, repräsentiert als Liste von Listen, zurückgeben. Die Reihenfolge, in der die Teilmengen angegeben werden, ist irrelevant, es darf sich in der Rückgabeliste aber kein Element wiederholen, weder in der äußeren, noch in einer der inneren Listen.

Beispiel: Die Eingabe `[1,2,3]` kann z.B. die Rückgabe

`[[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]]`

liefern.

Präsenzaufgabe 6:

Die Collatz-Folge beginnt mit einer positiven ganzen Zahl n und wird durch die folgenden Regeln erzeugt:

- Wenn n gerade ist, ist die nächste Zahl in der Folge $n/2$.
- Wenn n ungerade ist, ist die nächste Zahl in der Folge $3 * n + 1$.

Schreiben Sie eine Funktion `collatz`, welche eine positive ganze Zahl erwartet und rekursiv die Collatz-Folge, bis einschließlich zum ersten Auftreten einer 1, berechnet und auf der Konsole ausgibt. Der Rückgabewert der Funktion soll `None` sein.

Hinweis: Es ist bisher nicht bekannt, ob die Collatz-Folge für jede natürliche Zahl n irgendwann 1 erreicht. Da dies jedoch zumindest für alle $n \leq 2^{68}$ und somit für alle auf einem 64-Bit System darstellbaren Integer bewiesen ist, dürfen Sie annehmen, dass eine, nach obiger Aufgabenstellung, korrekt programmierte Rekursion terminiert.

Beispiel: Führt man `collatz(5)` aus, so sollten 5, 16, 8, 4, 2, 1 ausgegeben werden. Die Zahlen müssen in dieser Reihenfolge auf der Konsole ausgegeben werden, dürfen aber in verschiedenen Zeilen stehen. Die ausgegebenen Zahlen sollen immer Integer sein, z.B. soll also 8 nicht als 8.0 ausgegeben werden.